Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Escribir el contrato OpenAPI y revisarlo entre equipos como proveedor y consumidor.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 01 de Noviembre de 2026.

Objetivo o finalidad.

En este caso, se aborda el enfoque metodológico API-first, estableciendo la importancia de diseñar y formalizar las interfaces de programación antes de iniciar la etapa de programación en el servidor.

Por ello, a través de contratos estructurados utilizando OpenAPI 3.1, se definen de manera unívoca los recursos, las rutas, los métodos HTTP, los esquemas de datos y los códigos de respuesta, garantizando que el diseño sea comprensible tanto para el proveedor como para el consumidor de los servicios.

Es por eso, que se enfatiza la validación técnica mediante el uso de herramientas de análisis sintáctico (linters) y la revisión cruzada de especificaciones para mitigar ambigüedades, unificar el manejo de errores y permitir un desarrollo en paralelo eficiente y desacoplado entre los sistemas backend y las aplicaciones cliente.

1.- Documento de requisitos y reglas de negocio del contexto de la guía.

Descripción del problema: El sistema requiere gestionar de forma automatizada el control de préstamos de recursos y equipos a los usuarios, validando disponibilidad y límites de asignación mediante una API.

Recursos identificados: Usuario, Equipo, Préstamo.

Reglas de negocio principales:

RN-01: El usuario debe existir.

RN-02: El usuario debe estar activo.

RN-03: El usuario puede tener como máximo dos préstamos activos.

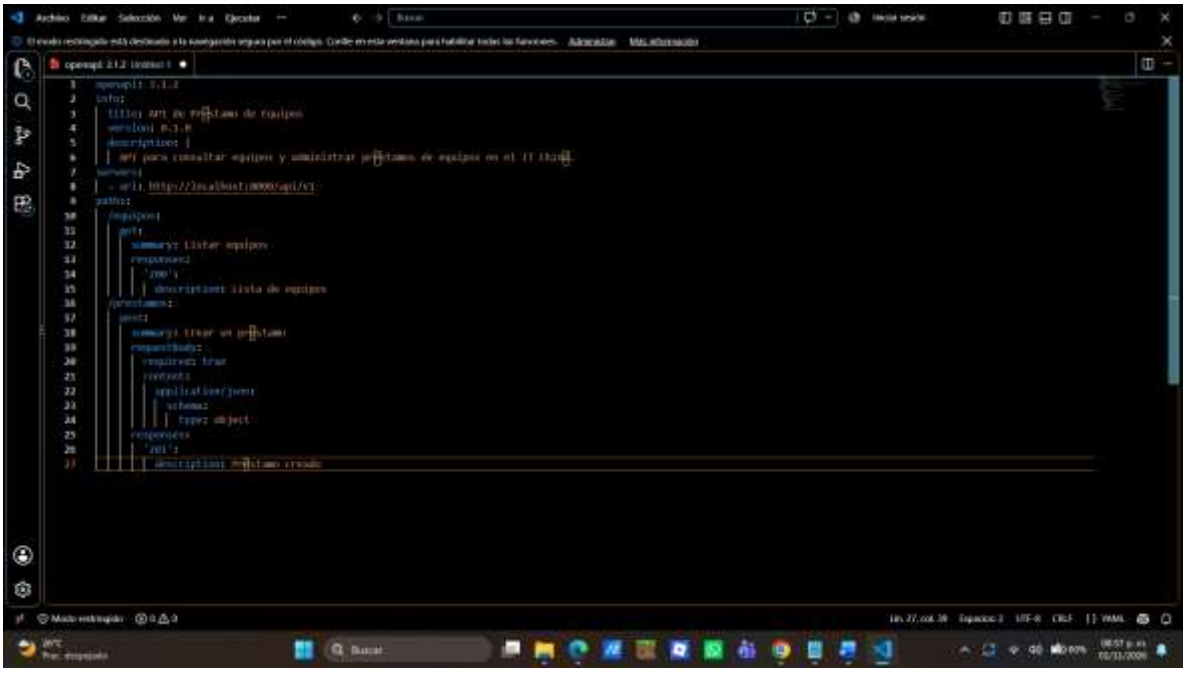
RN-04: El equipo debe existir.

RN-05: El equipo debe estar disponible.

RN-06: Cuando se crea el préstamo, el equipo cambia a "prestado".

RN-07: Cuando se devuelve el equipo, el préstamo cambia a "devuelto" y el equipo vuelve a "disponible".

2.- Archivo openapi.yaml inicial (Borrador / v0.1).

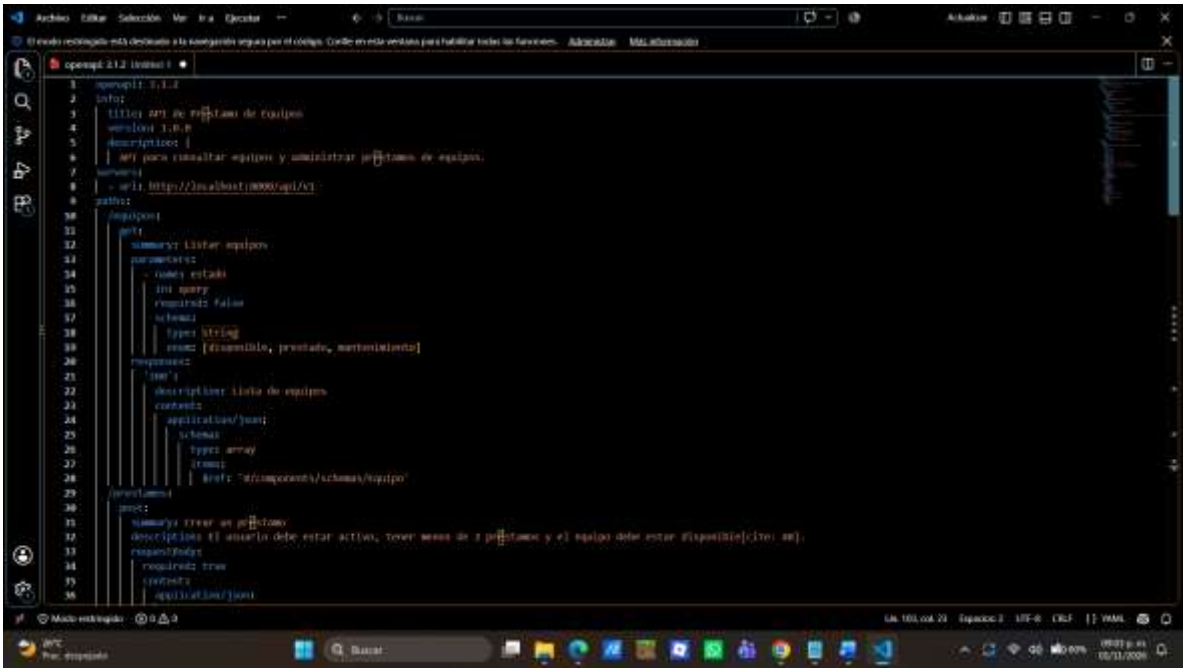


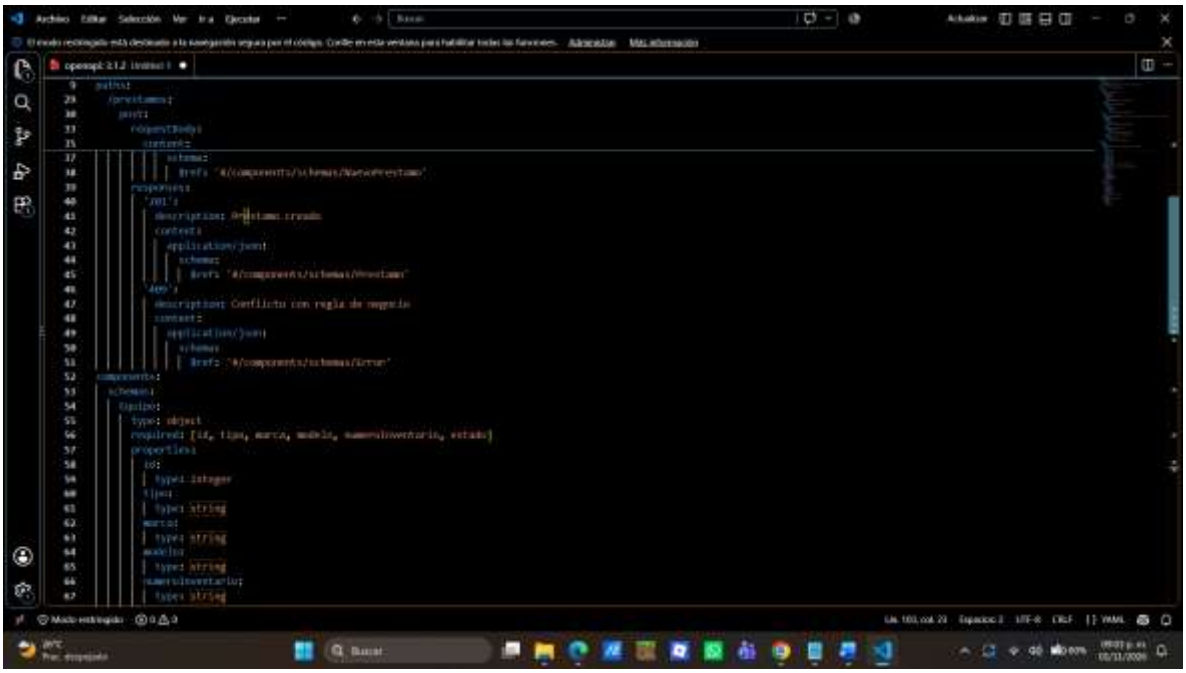
3.- Bitácora de revisión realizada por el equipo consumidor.

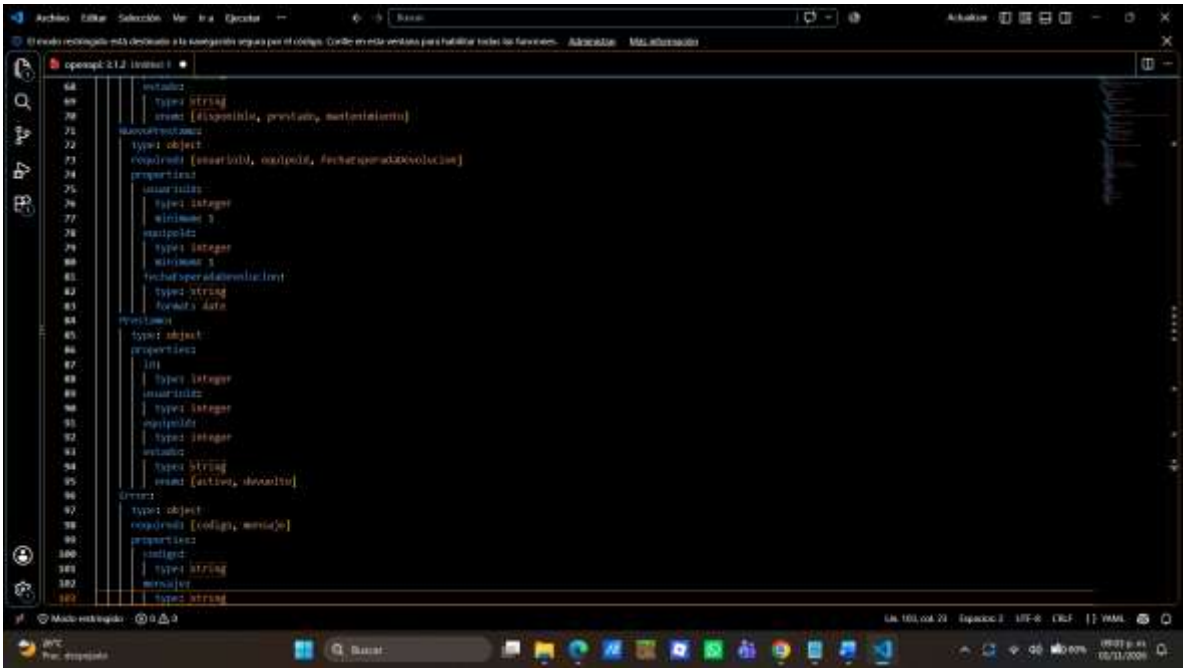
ID	Elemento	Problema encontrado.	Severidad.
OBS-01.	POST /prestamos.	No indica campos obligatorios en el cuerpo de la petición.	Alta
OBS-02.	Equipo.estado.	Valores de estado ambiguos (admite cualquier texto tipo string).	Media.
OBS-03.	Errores generales.	No existe un formato de respuesta uniforme para los errores.	Alta.

OBS-04.	Fechas.	No indica el formato esperado para las fechas de devolución.	Media.
OBS-05.	/equipos/{id}	No documenta la respuesta de error 404 cuando el equipo no existe.	Alta

4.- Archivo openapi.yaml corregido y aprobado (v1.0.0).







5.- Breve defensa de las decisiones tomadas.

Adopción de enum en estados: Se aceptó la observación del consumidor para evitar ambigüedades con cadenas de texto libres, estandarizando los estados a disponible, prestado y mantenimiento.

Implementación de esquema de errores común: Se creó un modelo de Error estandarizado (codigo y mensaje) para que el equipo de frontend (Flutter) maneje las excepciones de forma uniforme ante códigos 400, 404 o 409.

Definición estricta de restricciones: Se incorporaron validaciones como minimum: 1 en IDs y format: date en las fechas para asegurar la integridad de los datos antes de que lleguen al servidor de backend.

Respondiendo las preguntas de la página 31.

1.- ¿Qué observaciones recibió?

Se recibieron cinco observaciones técnicas por parte del equipo consumidor (OBS-01 a OBS-05), relacionadas con la falta de campos obligatorios en el requestBody, ambigüedad en los valores del atributo estado, ausencia de un formato uniforme de errores, falta de especificación en el formato de fechas y la omisión del código de error 404 en la consulta de equipos individuales.

2.- ¿Cuáles aceptó?

Se aceptaron en su totalidad las observaciones OBS-01, OBS-02, OBS-03, OBS-04 y OBS-05 para mejorar la precisión del contrato.

3.- ¿Cuáles rechazó?

No se rechazó ninguna observación en esta iteración, ya que todas aportaban claridad técnica necesaria para la integración con el cliente (Flutter).

4.- ¿Qué cambió?

Se actualizó el archivo openapi.yaml agregando la propiedad required en los cuerpos de petición, restricciones mediante enum para los estados permitidos (disponible, prestado, mantenimiento), el formato de fecha estándar (format: date), y un esquema de errores reutilizable para los códigos HTTP 400, 404 y 409.

5.- ¿Por qué?

Porque un contrato ambiguo obliga al desarrollador del cliente a adivinar cómo estructurar las peticiones o interpretar las respuestas, rompiendo el propósito fundamental del enfoque API-first.

Respondiendo las preguntas de la página 32.

1.- ¿El consumidor podría utilizar esta API sin hablar con ustedes?

Sí, porque el archivo openapi.yaml documenta de forma explícita las rutas, métodos HTTP, esquemas de entrada y códigos de respuesta, eliminando la necesidad de explicaciones verbales.

2.- ¿Cómo sabe el consumidor qué propiedades son obligatorias?

Mediante la directiva required: declarada dentro de los componentes y esquemas del contrato OpenAPI.

3.- ¿Cómo sabe qué valores puede contener estado?

Gracias al uso de la cláusula enum: que delimita los valores válidos (por ejemplo: disponible, prestado, mantenimiento).

4.- ¿Qué diferencia existe entre un 400 y un 409 en su API?

El código 400 Bad Request indica que la petición está mal formada o faltan datos sintácticos en el JSON; mientras que el 409 Conflict indica que la estructura es correcta pero la acción choca con una regla de negocio del sistema (por ejemplo, el equipo ya no está disponible o el usuario alcanzó su límite de préstamos).

5.- ¿Cómo sabe qué JSON debe enviar?

A través del objeto requestBody vinculado al esquema correspondiente (como NuevoPrestamo) y respaldado por ejemplos prácticos (examples) dentro del contrato.

6.- ¿Qué pasa si consulta un recurso inexistente?

El servidor responderá con un código HTTP 404 Not Found utilizando el esquema de error estándar definido en los componentes.

7.- ¿Todos sus errores tienen la misma estructura?

Sí, todos los errores comparten un esquema común denominado Error que contiene de forma uniforme los campos código y mensaje.

8.- ¿El nombre del endpoint representa un recurso o una acción?

Representa un recurso en forma de sustantivo plural (por ejemplo, /equipos o /prestamos), delegando la acción enteramente al método HTTP (GET, POST, PATCH), tal como dictan las buenas prácticas REST.

9.- ¿Están documentando solamente el caso exitoso o también los errores?

Se documentan ambos; tanto los códigos de éxito (200 OK, 201 Created) como las respuestas ante fallos y conflictos (400, 404, 409).

10.- Si mañana desarrollara el cliente en Flutter otra persona, ¿el contrato sería suficiente?

Sí, porque el contrato actúa como la fuente única de verdad y un puente formal completamente desacoplado de la implementación en el servidor.

Para el caso de nuestra API, para practicas agropecuarias.

1.- Documento de requisitos y reglas de negocio

Nombre del Proyecto: API de Prácticas Agropecuarias.

Objetivo: Proveer una arquitectura de servicios web RESTful y un backend robusto para la administración de parcelas agrícolas, tipos de cultivos y el registro digital de prácticas de campo realizadas por estudiantes y docentes, garantizando la persistencia incluso sin conexión a la red.

Reglas de Negocio Clave:

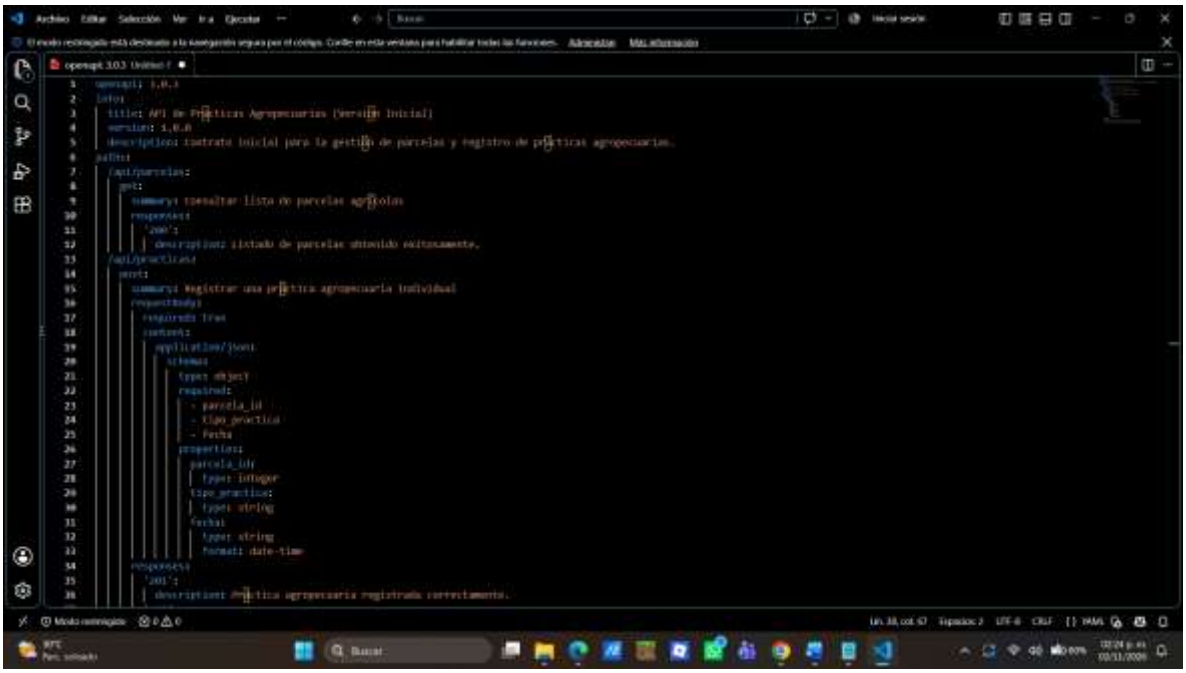
RN-01 (Registro Local / Offline): El cliente móvil debe permitir almacenar localmente las prácticas agropecuarias y observaciones de campo cuando el dispositivo no disponga de señal de internet.

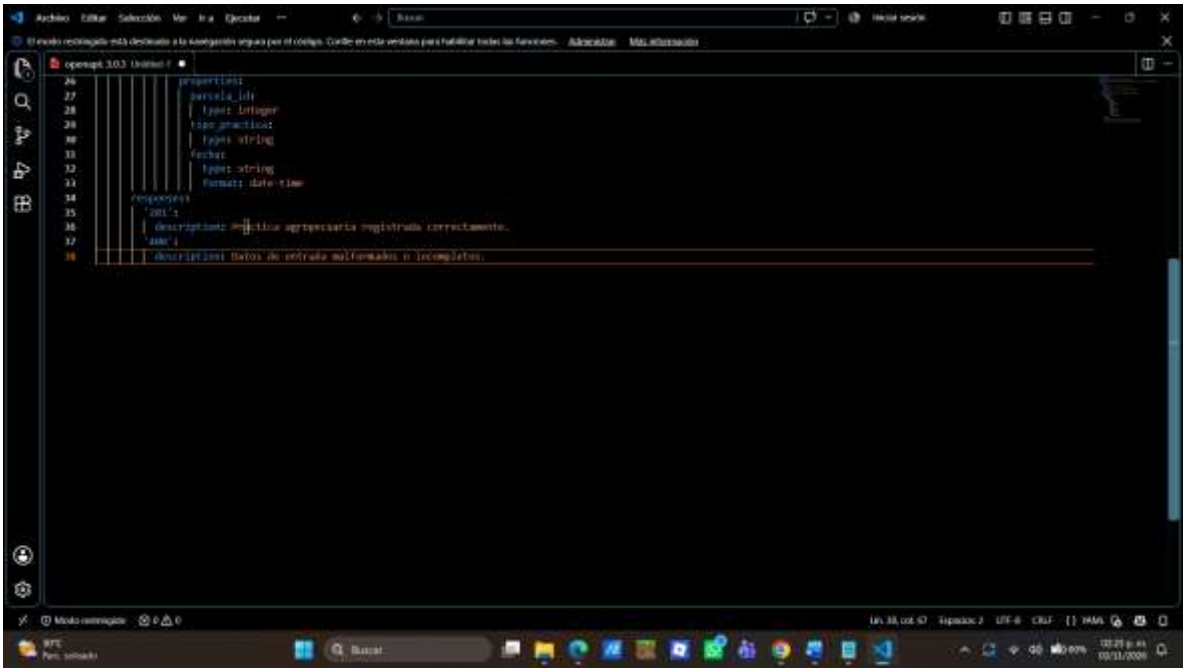
RN-02 (Sincronización por Lotes): Al restablecerse la conectividad, el sistema debe permitir enviar un bloque consolidado (batch) de las prácticas acumuladas en campo.

RN-03 (Asociación Estricta): Toda práctica agropecuaria registrada debe estar vinculada obligatoriamente a una parcela y un cultivo válidos dentro del sistema.

RN-04 (Estandarización de Errores): Las respuestas ante fallos de validación o conflictos de datos deben seguir un formato JSON uniforme con su respectivo código de estado HTTP.

2.- Archivo openapi.yaml inicial.



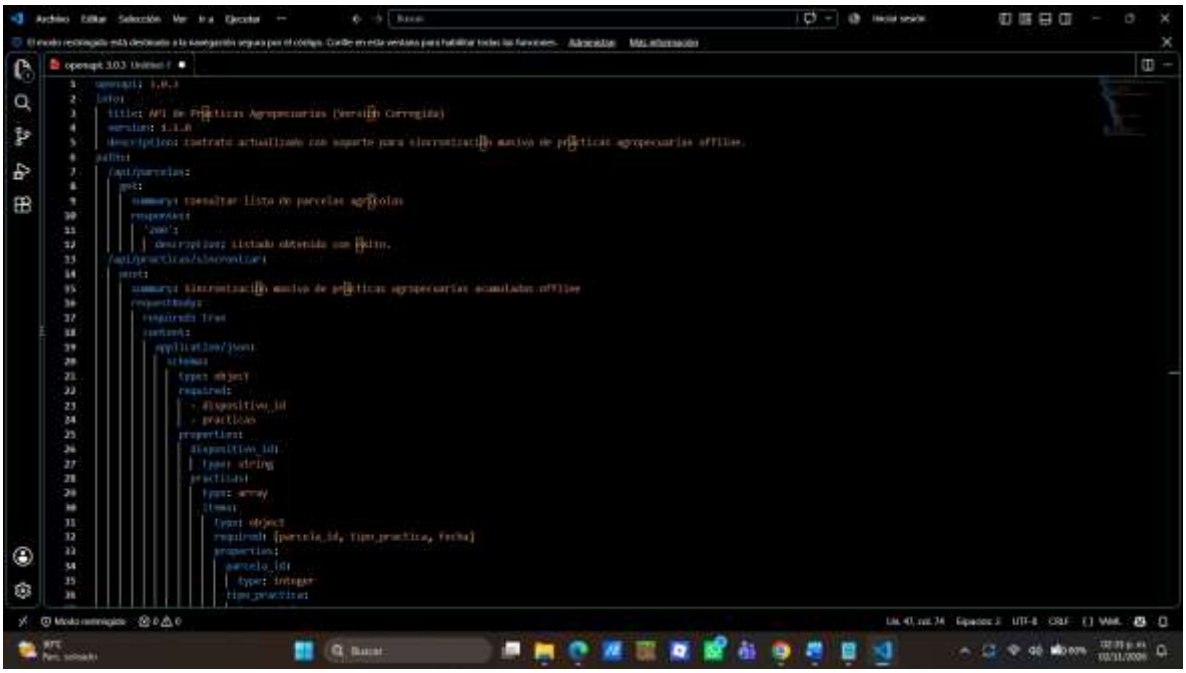


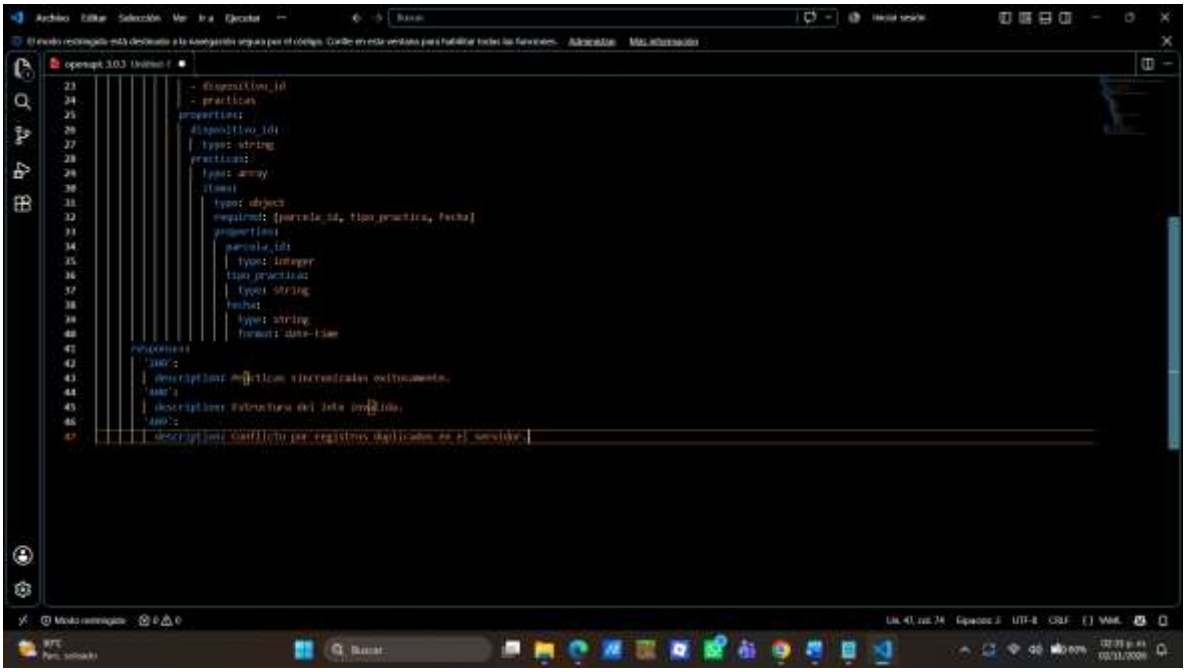
3.- Bitácora de revisión realizada por el equipo consumidor.

ID	Elemento.	Problema encontrado.	Severidad.	Propuesta.
OBS-01	POST /api/practicas.	No indica campos	Alta	Agregar la propiedad

		obligatorios en el cuerpo de la solicitud.		required en el esquema OpenAPI.
OBS-02	tipo_practica	Valores ambiguos o abiertos sin restricciones.	Media	Agregar un enum con los tipos permitidos (ej. riego, siembra, fertilización).
OBS-03	Estructura de errores.	No existe un formato de respuesta de error uniforme.	Alta	Crear un esquema de error estándar con código y mensaje.
OBS-04	Fechas (fecha)	No especifica el formato de fecha y hora.	Media	Utilizar el formato estándar date-time (ISO 8601).
OBS-05	Respuestas HTTP	No documenta la respuesta ante conflictos por duplicidad (409).	Alta	Agregar la respuesta 409 para registros de parcelas duplicadas.

4.- Archivo openapi.yaml corregido.





5.- Breve defensa de las decisiones tomadas.

Se toma en cuenta que para manejar y entender esto, se planteó el diseño de la API bajo el paradigma RESTful resolviendo una necesidad crítica del sector agropecuario: la conectividad intermitente.

Al proponer un contrato basado en OpenAPI con un mecanismo de sincronización por lotes (/api/practicas/sincronizar), se garantiza que el cliente móvil pueda recolectar información en campo sin pérdida de datos, por eso mismo, el uso riguroso de códigos HTTP (201, 400, 409) permite un acoplamiento limpio y predecible entre el servidor y cualquier interfaz cliente futura.

Ahora las preguntas de la página 32.

1.- ¿El consumidor podría utilizar esta API sin hablar con ustedes?

Sí, el contrato OpenAPI (openapi.yaml) detalla las rutas, los verbos HTTP permitidos y las estructuras de datos esperadas, lo que permite al equipo consumidor interactuar con los endpoints de parcelas y prácticas sin necesidad de comunicación directa adicional.

2.- ¿Cómo sabe el consumidor qué propiedades son obligatorias?

Mediante la sección required dentro de los esquemas del contrato OpenAPI, donde se enlistan explícitamente los campos indispensables (como parcela_id, tipo_practica y fecha).

3.- ¿Cómo sabe qué valores puede contener estado?

A través de la definición de restricciones de tipo y validaciones o listados de valores permitidos (enum) en el contrato para campos como el tipo de práctica agropecuaria.

4.- ¿Qué diferencia existe entre un 400 y un 409 en su API?

El código 400 (Bad Request) se utiliza cuando el cliente envía datos malformados o faltantes (por ejemplo, un JSON incompleto); el código 409 (Conflict) se emplea cuando la solicitud entra en conflicto con el estado actual del servidor, como intentar registrar una parcela o práctica duplicada.

5.- ¿Cómo sabe qué JSON debe enviar?

Consultando el objeto requestBody y sus respectivos esquemas modelados en el archivo OpenAPI, los cuales especifican el tipo de contenido (application/json) y la estructura de los atributos.

6.- ¿Qué pasa si consulta un recurso inexistente?

La API debe retornar un código de estado HTTP 404 Not Found acompañado de un mensaje JSON estandarizado que indique que el recurso solicitado no existe.

7.- ¿Todos sus errores tienen la misma estructura?

Sí, se implementa un esquema de error unificado que incluye el código de estado, un identificador de error legible y un mensaje descriptivo para facilitar la depuración en el cliente.

8.- ¿El nombre del endpoint representa un recurso o una acción?

Representa recursos orientados a objetos (sustantivos en plural como /api/parcelas y /api/practicas), utilizando los verbos HTTP (GET, POST) para definir la acción a realizar sobre ellos.

9.- ¿Están documentando solamente el caso exitoso o también los errores?

Se documentan ambos; el contrato especifica tanto las respuestas exitosas (200 OK, 201 Created) como las respuestas de error comunes (400, 409, 404).

10.- Si mañana desarrollara el cliente en Flutter otra persona, ¿el contrato sería suficiente?

Sí, porque el archivo openapi.yaml define con precisión matemática el contrato de comunicación cliente-servidor, permitiendo implementar la aplicación móvil en Flutter y la lógica de sincronización offline sin ambigüedades.